SORTING ALGORITHM TIME COMPARISON

Bubble Sort | Selection Sort | Insertion Sort | Merge Sort | Quick Sort

DAA Practical Experiment

Experiment	Runtime comparison of sorting algorithms
Input sizes	1000, 2000, 5000, 7000, 10000
Timing method	C clock() and CLOCKS_PER_SEC
Graphing	Python + Matplotlib
Heap Sort	Excluded as requested

This report contains the complete C benchmarking program, the complete Python graphing program, the recorded terminal output, and the generated comparison graph.

1. Experiment Overview

The C program generates one random input array for each input size, copies that same input into separate arrays, runs each sorting algorithm, measures only the sorting portion with clock(), prints the measured times, and writes the results to sorting_times.csv.

The Python program reads sorting_times.csv with csv.DictReader and plots all five algorithms against input size. The source uses the same labels and columns that appear in the CSV file.

Algorithm	Typical / Average Complexity	Measured in this run
Bubble Sort	O(n^2)	0.002, 0.008, 0.037, 0.064, 0.180 s
Selection Sort	O(n^2)	0.001, 0.001, 0.011, 0.026, 0.063 s
Insertion Sort	O(n^2) average	0.000, 0.002, 0.010, 0.020, 0.069 s
Merge Sort	O(n log n)	0.001, 0.001, 0.002, 0.004, 0.004 s
Quick Sort	O(n log n) average	0.000, 0.000, 0.000, 0.000, 0.002 s

Observed values above are transcribed from the supplied terminal screenshot. Tiny values such as 0.000000 are limited by the timer resolution and should not be interpreted as literally zero computation.

2. Complete C Benchmarking Code

Complete source code from the uploaded file, shown with line numbers for easier reference.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

/* ---------------- BUBBLE SORT ---------------- */

void bubbleSort(int arr[], int size)
{
    for (int i = 0; i < size - 1; i++)
    {
        for (int j = 0; j < size - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

/* ---------------- SELECTION SORT ---------------- */

void selectionSort(int arr[], int size)
{
    for (int i = 0; i < size - 1; i++)
    {
        int minIndex = i;

        for (int j = i + 1; j < size; j++)
        {
            if (arr[j] < arr[minIndex])
            {
                minIndex = j;
            }
        }

        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}

/* ---------------- INSERTION SORT ---------------- */

void insertionSort(int arr[], int size)
{
    for (int i = 1; i < size; i++)
    {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
```

2. Complete C Benchmarking Code - continued

```
    }
}

/* ---------------- MERGE SORT ---------------- */

void merge(int arr[], int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int *L = malloc(n1 * sizeof(int));
    int *R = malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++)
    {
        L[i] = arr[left + i];
    }

    for (int i = 0; i < n2; i++)
    {
        R[i] = arr[mid + 1 + i];
    }

    int i = 0;
    int j = 0;
    int k = left;

    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k] = L[i];
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }

        k++;
    }

    while (i < n1)
    {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2)
    {
        arr[k] = R[j];
        j++;
        k++;
    }

    free(L);
    free(R);
}
```

2. Complete C Benchmarking Code - continued

```
121
122 void mergeSort(int arr[], int left, int right)
123 {
124     if (left < right)
125     {
126         int mid = left + (right - left) / 2;
127
128         mergeSort(arr, left, mid);
129         mergeSort(arr, mid + 1, right);
130
131         merge(arr, left, mid, right);
132     }
133 }
134
135 /* ---------------- QUICK SORT ---------------- */
136
137 int partition(int arr[], int low, int high)
138 {
139     int pivot = arr[high];
140
141     int i = low - 1;
142
143     for (int j = low; j < high; j++)
144     {
145         if (arr[j] < pivot)
146         {
147             i++;
148
149             int temp = arr[i];
150             arr[i] = arr[j];
151             arr[j] = temp;
152         }
153     }
154
155     int temp = arr[i + 1];
156     arr[i + 1] = arr[high];
157     arr[high] = temp;
158
159     return i + 1;
160 }
161
162 void quickSort(int arr[], int low, int high)
163 {
164     if (low < high)
165     {
166         int pi = partition(arr, low, high);
167
168         quickSort(arr, low, pi - 1);
169         quickSort(arr, pi + 1, high);
170     }
171 }
172
173 /* ---------------- COPY ARRAY ---------------- */
174
175 void copyArray(int source[], int destination[], int size)
176 {
177     for (int i = 0; i < size; i++)
178     {
179         destination[i] = source[i];
180     }
```

2. Complete C Benchmarking Code - continued

```
}

/* ---------------- MAIN ---------------- */

int main()
{
    /*
       Different input sizes.
       These are deliberately kept reasonable because
       Bubble, Selection and Insertion become slow as n grows.
    */

    int sizes[] = {1000, 2000, 5000, 7000, 10000};

    int numberOfSizes = 5;

    FILE *file = fopen("sorting_times.csv", "w");

    if (file == NULL)
    {
        printf("Could not create CSV file.\n");
        return 1;
    }

    fprintf(file, "Size,Bubble,Selection,Insertion,Merge,Quick\n");

    srand(time(NULL));

    for (int s = 0; s < numberOfSizes; s++)
    {
        int size = sizes[s];

        printf("\nTesting size = %d\n", size);

        int *original = malloc(size * sizeof(int));
        int *arr1 = malloc(size * sizeof(int));
        int *arr2 = malloc(size * sizeof(int));
        int *arr3 = malloc(size * sizeof(int));
        int *arr4 = malloc(size * sizeof(int));
        int *arr5 = malloc(size * sizeof(int));

        if (original == NULL || arr1 == NULL || arr2 == NULL ||
            arr3 == NULL || arr4 == NULL || arr5 == NULL)
        {
            printf("Memory allocation failed.\n");

            free(original);
            free(arr1);
            free(arr2);
            free(arr3);
            free(arr4);
            free(arr5);

            fclose(file);
            return 1;
        }

        /* Generate same random input for all algorithms */

        for (int i = 0; i < size; i++)
```

2. Complete C Benchmarking Code - continued

```
241         {
242             original[i] = rand() % 1000000;
243         }
244 
245         /* ---------------- BUBBLE ---------------- */
246 
247         copyArray(original, arr1, size);
248 
249         clock_t start = clock();
250 
251         bubbleSort(arr1, size);
252 
253         clock_t end = clock();
254 
255         double bubbleTime =
256             (double)(end - start) / CLOCKS_PER_SEC;
257 
258         /* ---------------- SELECTION ---------------- */
259 
260         copyArray(original, arr2, size);
261 
262         start = clock();
263 
264         selectionSort(arr2, size);
265 
266         end = clock();
267 
268         double selectionTime =
269             (double)(end - start) / CLOCKS_PER_SEC;
270 
271         /* ---------------- INSERTION ---------------- */
272 
273         copyArray(original, arr3, size);
274 
275         start = clock();
276 
277         insertionSort(arr3, size);
278 
279         end = clock();
280 
281         double insertionTime =
282             (double)(end - start) / CLOCKS_PER_SEC;
283 
284         /* ---------------- MERGE ---------------- */
285 
286         copyArray(original, arr4, size);
287 
288         start = clock();
289 
290         mergeSort(arr4, 0, size - 1);
291 
292         end = clock();
293 
294         double mergeTime =
295             (double)(end - start) / CLOCKS_PER_SEC;
296 
297         /* ---------------- QUICK ---------------- */
298 
299         copyArray(original, arr5, size);
300 
```

2. Complete C Benchmarking Code - continued

```
        start = clock();

        quickSort(arr5, 0, size - 1);

        end = clock();

        double quickTime =
            (double)(end - start) / CLOCKS_PER_SEC;

        /* Print results */

        printf("Bubble    : %f seconds\n", bubbleTime);
        printf("Selection : %f seconds\n", selectionTime);
        printf("Insertion : %f seconds\n", insertionTime);
        printf("Merge     : %f seconds\n", mergeTime);
        printf("Quick     : %f seconds\n", quickTime);

        /* Save to CSV */

        fprintf(file, "%d,%f,%f,%f,%f,%f\n",
                size,
                bubbleTime,
                selectionTime,
                insertionTime,
                mergeTime,
                quickTime);

        free(original);
        free(arr1);
        free(arr2);
        free(arr3);
        free(arr4);
        free(arr5);
    }

    fclose(file);

    printf("\nResults saved to sorting_times.csv\n");

    return 0;
}
```

3. Complete Python Graphing Code

Complete source code from the uploaded file, shown with line numbers for easier reference.

```
 1  import csv
 2  import matplotlib.pyplot as plt
 3
 4  sizes = []
 5
 6  bubble = []
 7  selection = []
 8  insertion = []
 9  merge = []
10  quick = []
11
12  with open("sorting_times.csv", "r") as file:
13
14      reader = csv.DictReader(file)
15
16      for row in reader:
17
18          sizes.append(int(row["Size"]))
19
20          bubble.append(float(row["Bubble"]))
21          selection.append(float(row["Selection"]))
22          insertion.append(float(row["Insertion"]))
23          merge.append(float(row["Merge"]))
24          quick.append(float(row["Quick"]))
25
26
27  plt.plot(sizes, bubble, marker='o', label="Bubble Sort")
28  plt.plot(sizes, selection, marker='o', label="Selection Sort")
29  plt.plot(sizes, insertion, marker='o', label="Insertion Sort")
30  plt.plot(sizes, merge, marker='o', label="Merge Sort")
31  plt.plot(sizes, quick, marker='o', label="Quick Sort")
32
33  plt.xlabel("Input Size")
34  plt.ylabel("Time (seconds)")
35
36  plt.title("Sorting Algorithm Time Comparison")
37
38  plt.legend()
39
40  plt.grid(True)
41
42  plt.show()
```

4. Program Output Screenshot

This is the supplied VS Code / PowerShell output produced by the C benchmarking program.

```
PS C:\Users\sahil\OneDrive\Dokumen\DAA_programs> cd 'c:\Users\sahil\OneDrive\Dokumen\DAA_programs\output'
PS C:\Users\sahil\OneDrive\Dokumen\DAA_programs\output> & .\'time_comparison_sorting.exe'

Testing size = 1000
Bubble    : 0.002000 seconds
Selection : 0.001000 seconds
Insertion : 0.000000 seconds
Merge     : 0.001000 seconds
Quick     : 0.000000 seconds

Testing size = 2000
Bubble    : 0.008000 seconds
Selection : 0.001000 seconds
Insertion : 0.002000 seconds
Merge     : 0.001000 seconds
Quick     : 0.000000 seconds

Testing size = 5000
Bubble    : 0.037000 seconds
Selection : 0.011000 seconds
Insertion : 0.010000 seconds
Merge     : 0.002000 seconds
Quick     : 0.000000 seconds

Testing size = 7000
Bubble    : 0.064000 seconds
Selection : 0.026000 seconds
Insertion : 0.020000 seconds
Merge     : 0.004000 seconds
Quick     : 0.000000 seconds

Testing size = 10000
Bubble    : 0.180000 seconds
Selection : 0.063000 seconds
Insertion : 0.069000 seconds
Merge     : 0.004000 seconds
Quick     : 0.002000 seconds
```

Figure 1. Terminal output showing measured times for all five sorting algorithms.

5. Sorting Time Comparison Graph

The supplied graph plots input size on the x-axis and measured execution time in seconds on the y-axis. The three quadratic algorithms rise more rapidly than Merge Sort and Quick Sort in the shown experiment.

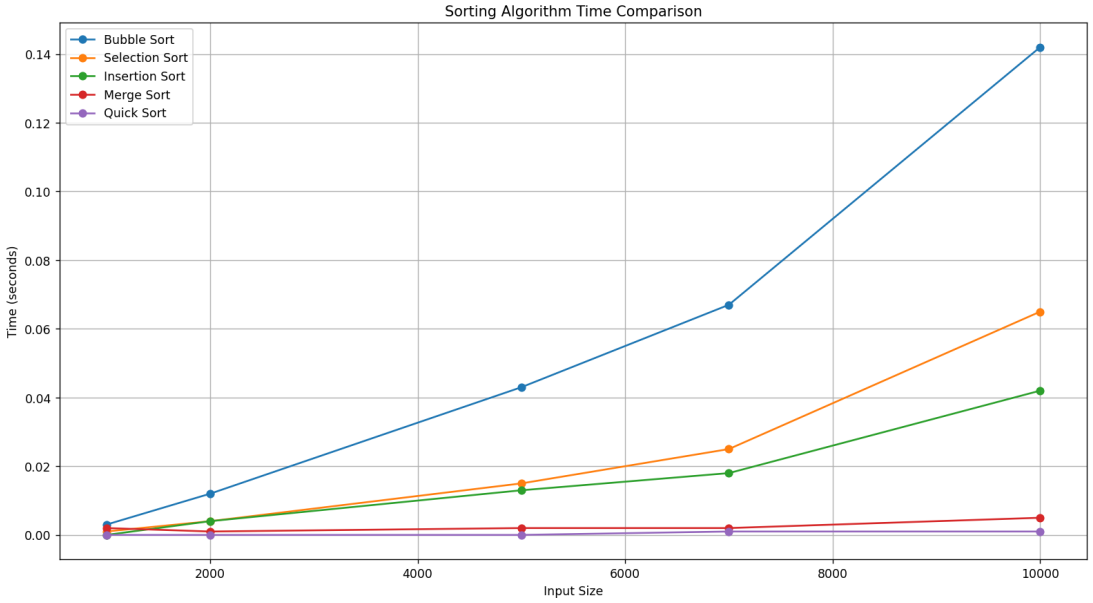

Figure 2. Sorting Algorithm Time Comparison generated with Python and Matplotlib.

6. Result and Conclusion

For the measured input sizes, Bubble Sort shows the largest increase in runtime. Selection Sort and Insertion Sort also grow substantially as the input increases. Merge Sort and Quick Sort remain much faster in the supplied run, which is consistent with their lower average asymptotic growth rates.

The exact timings are machine-dependent. The experiment is therefore most useful for observing the overall growth trend, rather than treating one individual timing value as universal.

The attached source files are preserved in the report as provided for the experiment: the C program benchmarks the five sorting algorithms and writes sorting_times.csv, while the Python program reads that CSV and creates the comparison graph.

Observed timing values from the supplied terminal screenshot (seconds):

Input Size	Bubble	Selection	Insertion	Merge	Quick
1000	0.002	0.001	0.000	0.001	0.000
2000	0.008	0.001	0.002	0.001	0.000
5000	0.037	0.011	0.010	0.002	0.000
7000	0.064	0.026	0.020	0.004	0.000
10000	0.180	0.063	0.069	0.004	0.002